RV University

School of Computer Science and Engineering

Mini Project involving India’s Sovereign Models

Kisan Sahayak: A Source-Grounded Farmer Advisory Assistant built on BharatGen AgriParam and AI4Bharat IndicTrans2

Project Report

Submitted by

Aadit Pani

USN: 1RVU23CSE002

Semester VII | Course: WWLLM

November 2026

Contents

1. Abstract
2. Introduction and Motivation
3. Objectives
4. India’s Sovereign Models Used
5. System Architecture
6. Implementation Details
7. Knowledge Base
8. Answering Strategy: Why Source-First
9. Evaluation and Results
10. Limitations
11. Future Work
12. Conclusion
13. References
14. Appendix: Running the Project

1. Abstract

This project builds Kisan Sahayak, an advisory assistant that answers Indian farmers' questions about wheat cultivation (sowing time, seed rate, fertilizer dose, irrigation, weeds, pests, diseases, harvest and storage). It is built around models developed in India: BharatGen's AgriParam, an agriculture-focused Param language model, for language generation, and AI4Bharat's IndicTrans2 for translation between English and Indian languages. Answers are grounded in ICAR extension documents through retrieval-augmented generation (RAG).

The central engineering finding is that a small 4-bit local model, when asked to paraphrase, was wrong often enough (24% of the time on our test sets) to be unsafe for agronomic advice. The final system therefore answers with the exact text of the best-matching source passage and its citation, uses the model only for an optional, strictly verified plain-language summary, and refuses questions it cannot ground. On a held-out set of 68 questions written before the guards were built, the system never answered an unanswerable question (0/35) and the quoted source held the correct fact in 29 of 33 answerable cases (88%).

2. Introduction and Motivation

Farmers need timely, locally relevant agronomic guidance, but extension services are stretched and most authoritative material is written in English, in long bulletins. General-purpose chatbots are fluent but can state wrong doses with confidence, and a wrong fertilizer or pesticide figure has a direct financial and agronomic cost. Two properties matter more than fluency here: answers must be traceable to an authoritative source, and the system must say "I don't know" when the sources are silent.

The project also asks a sovereignty question: how far can a useful, safe application be built on Indian-developed models that run locally, without sending farmer queries to a foreign cloud API? The whole stack here runs on a single laptop: the LLM is a 1.7 GB quantised model served by llama.cpp, the embedder and reranker are small, and translation runs on-device.

3. Objectives

- Answer wheat-farming questions in English and Indian languages using only vetted ICAR documents.
- Use Indian sovereign models (BharatGen AgriParam, AI4Bharat IndicTrans2) as core components.
- Cite the source passage and page for every answer.
- Refuse reliably for out-of-scope, unsupported or live-data questions (prices, weather, schemes, other crops).
- Run fully locally on commodity hardware (Apple Silicon, Metal GPU).
- Measure behaviour with an honest evaluation, including a held-out set written before tuning.

4. India's Sovereign Models Used

Component	Model / Provider	Role in the project
Generator LLM	AgriParam (BharatGen), 4-bit GGUF Q4_K_M, about 1.7 GB, 2048-token context	Drafts a short plain-language summary of the retrieved passage; served with llama.cpp on Metal

Translation	IndicTrans2 distilled 200M models (AI4Bharat): Indic-to-English and English-to-Indic	Translates the farmer's question into English and the answer back into Hindi, Odia, Kannada, Tamil, Telugu, Marathi, Bengali, Punjabi or Gujarati
Embeddings	intfloat/multilingual-e5-small	First-stage retrieval over document chunks (not Indian-developed)
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2	Second-stage relevance scoring and refusal signal (not Indian-developed)

AgriParam is a domain model from BharatGen, India's government-backed initiative for foundation models in Indian languages; the Param family targets Indian-language and Indian-context use. IndicTrans2 from AI4Bharat (IIT Madras) is a state-of-the-art open translation model across the 22 scheduled Indian languages. The embedding model and reranker are general open models and are noted as the non-sovereign parts of the stack.

5. System Architecture

The request path is a pipeline with explicit gates before any text is shown to the user:

Stage	What happens
1. Frontend	Static HTML/JS page; the user picks a language and asks a question.
2. Input normalisation	Empty or junk input and greetings return a help message. Devanagari and Roman-script Hindi ("Hinglish") are detected; Hinglish is rewritten into English with a word glossary.
3. Translate in	Non-English questions go through IndicTrans2 to English, after a small agronomy glossary fixes known mistranslations (e.g. khad to fertilizer, ratua to rust disease).
4. Retrieve	multilingual-e5 returns 8 candidate chunks by cosine similarity; the cross-encoder reranks them.
5. Gates	Refuse if reranker score is low, if another crop or live/commercial data is named, or if a key question word appears nowhere in the corpus and the reranker is not decisive.
6. Answer	The exact text of the best passage plus citation. AgriParam's summary is added only if it passes the numeric and vocabulary checks.
7. Translate out	For non-English users the source text is translated back; the unverified model summary is never translated.

Backend: FastAPI exposes /api/ask, /api/languages, /api/health and /api/reindex, and serves the frontend as static files. Configuration is read from a .env file with pydantic-settings. The LLM backend is pluggable: llama-server (OpenAI-compatible), in-process llama-cpp-python, or an extractive mode that needs no LLM.

6. Implementation Details

6.1 Code layout

File	Responsibility
backend/app/main.py	FastAPI app, endpoints, static frontend mount

backend/app/rag.py	Pipeline: input handling, retrieval, gates, source-first answer, guards
backend/app/retrieval.py	Chunking, e5 embedding index (numpy, cached in data/index.npz), cross-encoder reranking, unseen-term check
backend/app/llm.py	Prompt building and the three generator backends
backend/app/translate.py	IndicTrans2 wrapper, FLORES language codes, Hindi query glossary
backend/app/hinglish.py	Roman-script Hindi detection and word-for-word rewrite
backend/app/config.py	Settings and the language list
frontend/	index.html, style.css, app.js (no build step)
eval/	Test sets, run_eval.py and result CSVs

6.2 Retrieval

Documents are split into chunks of about 110 words with 20 words of overlap, small because AgriParam has a 2048-token window. Markdown sources are split per paragraph under their heading so that a table row is never cut or merged. Chunk embeddings are cached and rebuilt when a content fingerprint changes. Since multilingual-e5 scores are compressed into a narrow band (wheat passages score above 0.84 for almost any sowing question), the cross-encoder reranker is the main relevance signal: off-topic questions score strongly negative.

6.3 Gates and refusal

- **Reranker gate:** refuse when the top reranker score is below ce_min.
- **Scope gate:** the corpus is wheat only, so other crops named in English or Hindi (paddy, mustard, dhan, सरसों …) are refused unless wheat or crop-residue is also mentioned.
- **Live-data gate:** prices, mandi, weather, insurance, subsidy, loans and general-knowledge probes are refused because no agronomy bulletin contains them.
- **Vocabulary gate:** if a content word of the question appears nowhere in the corpus and the reranker is not decisive, refuse.
- **Prompt-injection and junk:** greetings, gibberish and “ignore your instructions” style inputs never reach the model as instructions and are answered with a fixed message.

6.4 Verifying the model’s summary

The AgriParam draft is shown only if it (a) contains no number, product name or month absent from the best passage, (b) is not an echo or a question, and (c) has at most 10% novel content words relative to the passage. A practical gotcha: AgriParam’s chat template drops the system role, so all instructions are placed in the single user turn.

6.5 Multilingual handling

AgriParam’s Hindi output was unreliable (it replied in English and invented facts), so only English is handled by the LLM. Hindi and all other languages go through IndicTrans2 in and out. Roman-script Hindi is handled by a glossary of about 200 words with a trigger requiring at least two glossary hits. IndicTrans2 needs transformers below version 5 and use_cache=False because its remote code breaks on newer cache classes.

7. Knowledge Base

Source	Details
Wheat Cultivation in India: Pocket Guide	Kumar, A. et al., Extension Bulletin 52, ICAR-Directorate of Wheat Research, Karnal, Sept 2014. English, 41 pages.
Cleaned tables (hand-written)	Tables 1–6 and the 4W-1H chart of the bulletin restated as self-contained sentences, because PDF extraction scrambles tables. Table 5 was checked against the page image; no numbers were added or changed.
Farmer's fertilizer practices in wheat	Tripathi, S.C., Kumar, N., Samota, S.R., Indian Farming 73(9): 37–39, Sept 2023. Karnal (Haryana) case study, regional supporting source.

Known gaps: no Hindi-language sources, no crops other than wheat, and no state-specific Uttar Pradesh package of practices. A focused corpus was chosen deliberately: a small, high-quality corpus beats a large noisy one.

8. Answering Strategy: Why Source-First

The first version let the model paraphrase the retrieved passages. Hand review showed frequent errors, for example a storage answer that gave 14% moisture and advised sprinkling water, where the bulletin says below 12%. Across the test sets the model's own wording was wrong 24% of the time, while the best retrieved passage held the right fact 95% of the time, and no guard setting made the model's text more accurate than the quote. The design decision follows from the data: the source quotation is the answer; the model is a verified, optional extra. This trades polish for trustworthiness, which is the right trade for agronomic advice.

9. Evaluation and Results

Two question sets were written in JSONL with fields for language, answerability, gold source and any-of keywords: a held-out set of 68 cases written before the guards and reranker existed (the honest number), and a tuning set of 66 cases used during development (optimistic). Both include other crops, topics the corpus lacks, prompt injection, greetings, junk, Hindi and Roman Hindi. eval/run_eval.py posts each case to the running API and reports retrieval hit@k, keyword recall, refusal accuracy per language and median latency.

Metric	Held-out (68)	Tuned set (66)
Retrieval hit@4 (answerable)	33/33 = 100%	43/44 = 98%
Source passage has the expected fact	29/33 = 88%	44/44 = 100%
Unanswerable questions answered anyway	0/35	0/22
Answerable questions refused	3/33	0/44
Hindi / Roman-Hindi answerable	8/8	11/11

Progression on the held-out set: first version 17/33 (52%); reranker plus calibrated gates 25/33 (76%); source-first answers 28/33 (85%); residue and live-data guard fixes 29/33 (88%).

Remaining failures (4 of 33 held-out)

- Wrongly refused: “only one irrigation, when?” (the table passage is phrased differently), “how much FYM gave the highest yield”, and Hindi “urea kab dalni chahiye”. A false refusal is safe but annoying.
- Related but not best passage: cost of production per quintal and “does zero tillage save money” returned a neighbouring passage.

Caveat: keyword match is only a proxy for correctness, so the result CSVs (eval/heldout_v4.csv, eval/results_final.csv) were also read by hand. The most important safety result, zero unanswerable questions answered, held on both sets.

10. Limitations

- Scope is wheat only, and mostly one region (UP wheat zones NWPZ, NEPZ, CZ); advice outside it is refused.
- Vocabulary lists and crop lists are hand-written and can misfire on unusual wording.
- Evaluation sets are small and written by the author; the held-out set was informed by earlier failures.
- Translated answers are the translated source quote and inherit machine-translation errors, such as spelling variants of chemical names.
- The in-process llama-cpp-python path failed to build on the development Mac; llama-server is the tested path.
- IndicTrans2 models are gated on Hugging Face and need an accepted licence and login.

11. Future Work

- Add Hindi and state-language extension material and more crops, each with its own held-out questions.
- Retrieve and answer directly in Indian languages using multilingual embeddings, once a sovereign model’s Hindi generation is reliable.
- Replace the hand-written crop and live-data lists with a learned scope classifier.
- Voice input and output for low-literacy users, and a WhatsApp or IVR front end.
- Larger evaluation with agronomist-reviewed answers and human rating of the plain-language summaries.

12. Conclusion

Kisan Sahayak shows that a safe, useful farmer advisory assistant can be assembled from Indian open models and run entirely on a laptop. The main lesson is that for high-stakes advice, grounding and refusal matter more than generation: measuring the small model’s error rate led to a source-first design that never answered an out-of-scope question on the held-out set and returned the correct fact in 88% of answerable cases, with every answer cited. Sovereign models contribute the language and domain layer; careful retrieval, gating and verification make them dependable.

13. References

1. BharatGen. AgriParam and the Param family of Indian foundation models. GGUF build: arunmcops/AgriParam-GGUF (Hugging Face).
2. Gala, J. et al. (AI4Bharat). IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages. TMLR, 2023.
3. Wang, L. et al. Text Embeddings by Weakly-Supervised Contrastive Pre-training (E5). 2022; multilingual-e5-small.
4. Kumar, A. et al. Wheat Cultivation in India: Pocket Guide. Extension Bulletin 52, ICAR-DWR, Karnal, 2014.
5. Tripathi, S.C., Kumar, N., Samota, S.R. Farmer's fertilizer practices in wheat. Indian Farming 73(9), 2023.
6. Lewis, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.
7. llama.cpp (ggml-org), FastAPI, sentence-transformers, Hugging Face Transformers.

Appendix: Running the Project

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
brew install llama.cpp
llama-server -m AgriParam-Q4_K_M.gguf -c 2048 -ngl 99 --port 8081 --alias agriparam
# .env: LLM_BACKEND=openai, OPENAI_BASE_URL=http://localhost:8081/v1, OPENAI_MODEL=agriparam
# optional: TRANSLATOR=indictrans2 (after hf auth login and accepting the gated model terms)
uvicorn backend.app.main:app --reload      # open http://127.0.0.1:8000
python eval/run_eval.py                    # evaluation
```